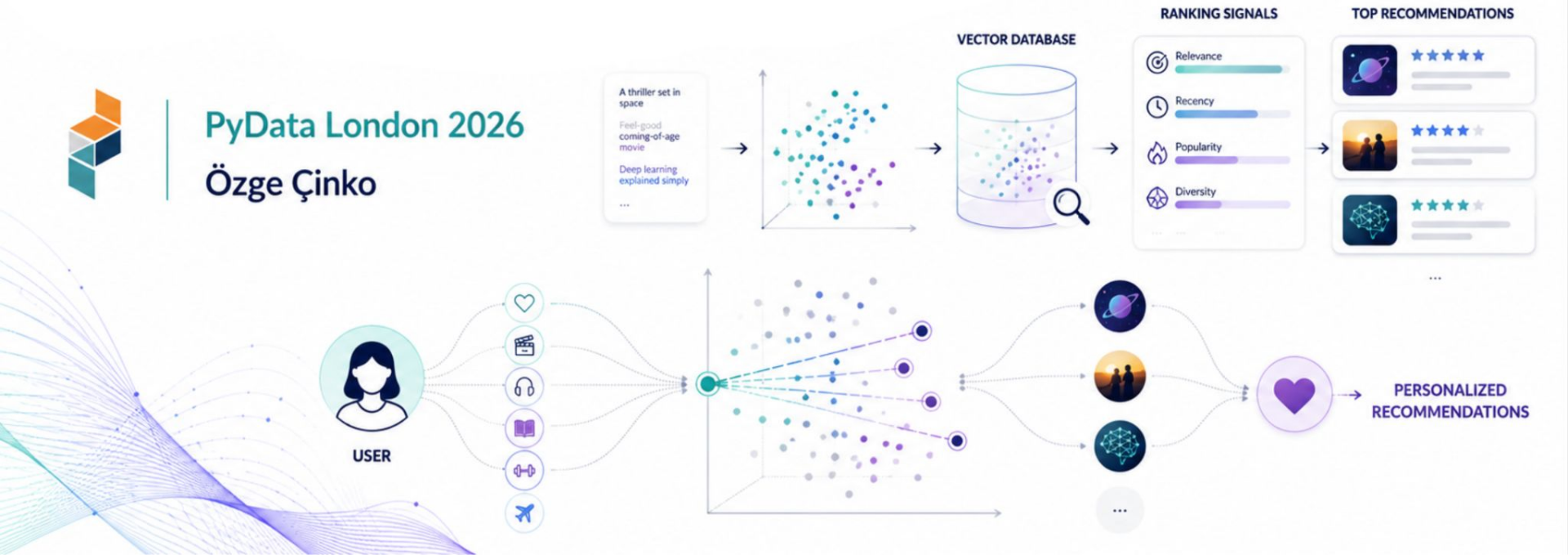
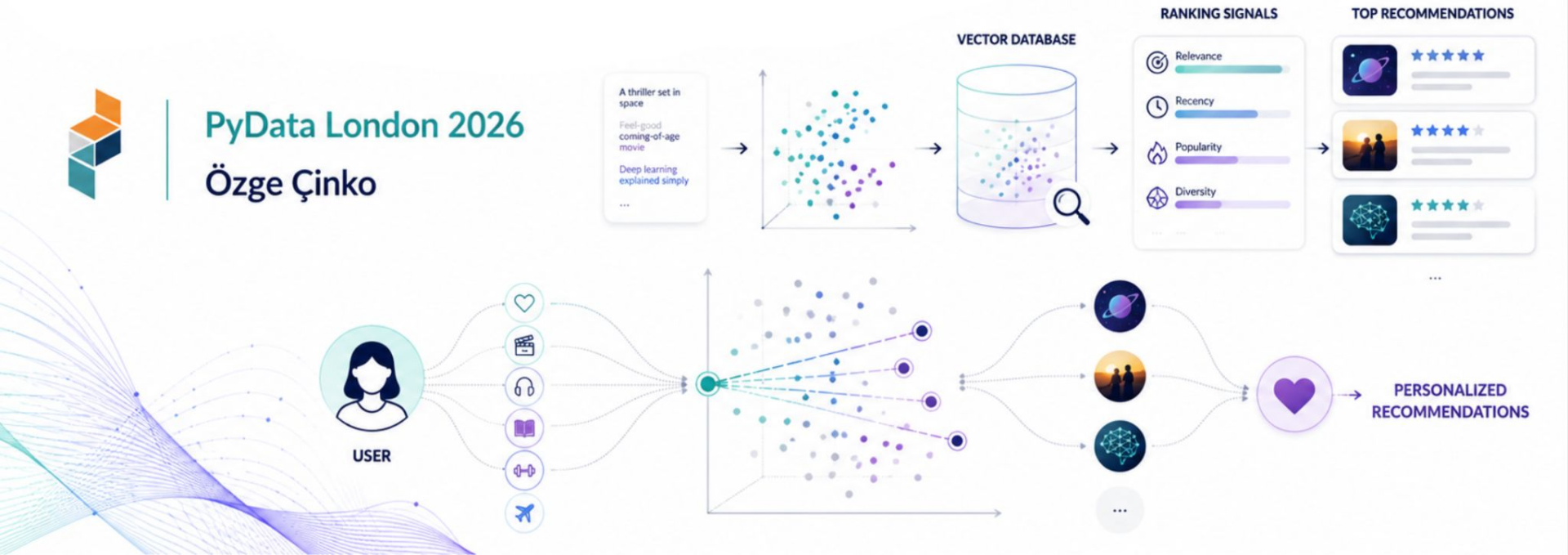
LLM-Based Recommendation Systems

From Embeddings to Real Personalization



LLM-Based Recommendation Systems

From Embeddings to Real Personalization

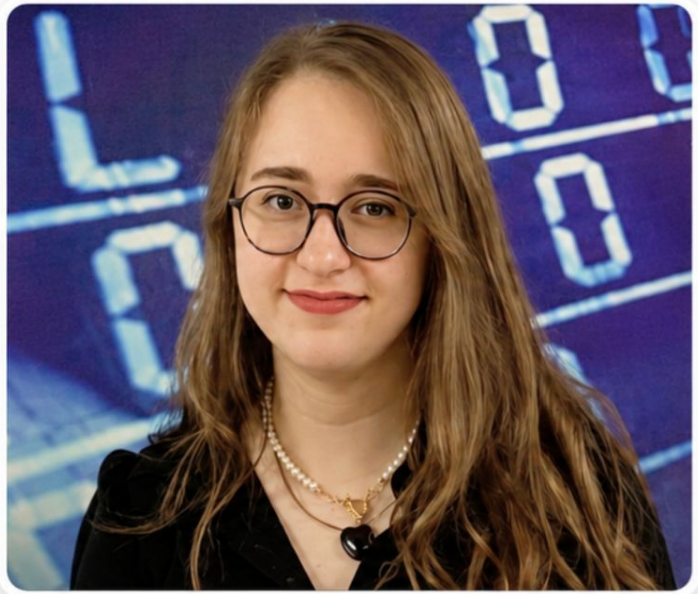


ABOUT ME

Who am I?

Now: AI Engineer at **ING**, working on agentic AI

Before: 2 years as AI Research Engineer at **Huawei**, including recommendation systems



Find me online

github.com/ozgecinko

linkedin.com/in/ozgecinko

ozgecinko.medium.com

Connect with me

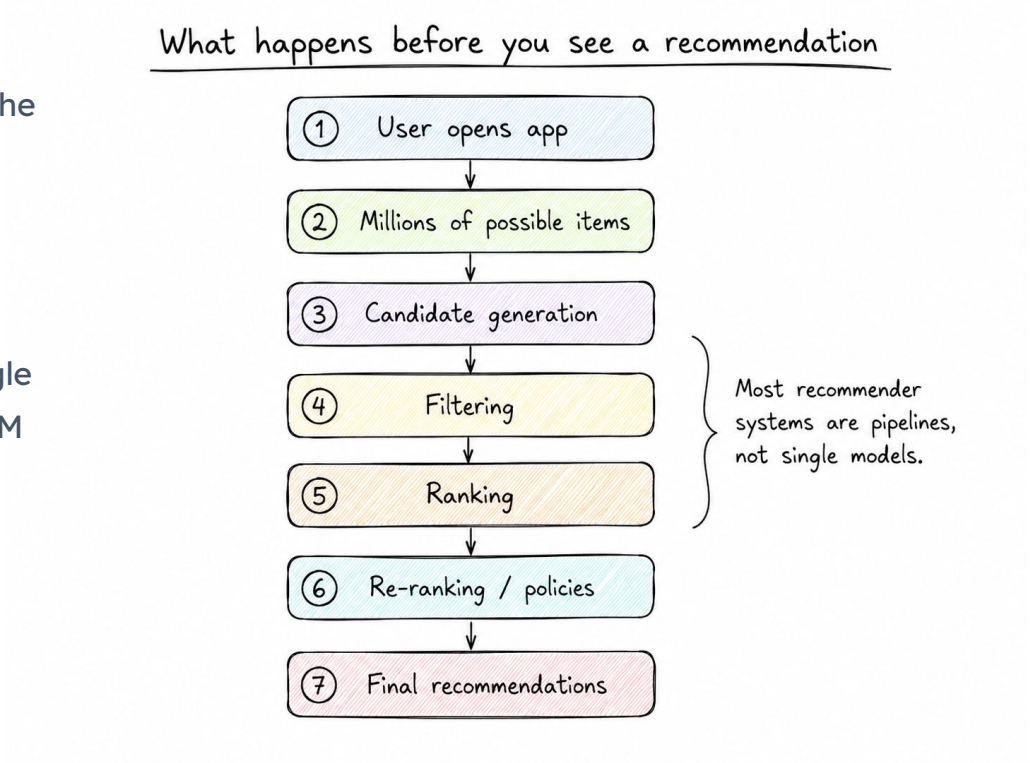
Feel free to reach out!

THE MACHINE YOU NEVER SEE

Almost every screen is a ranking decision

The next song, the autoplaying video, the products that follow you, the Netflix row at the top; each is a system that retrieves, filters, ranks and re-ranks millions of candidates in milliseconds.

Most recommenders are pipelines, not a single model. That structure is exactly where an LLM can be inserted or misused.



WHY IT MATTERS

Personalization is the business, not a feature

Two famous numbers: Amazon ≈ 1/3 of purchases, Netflix ≈ 3/4 of watch time — trace to a 2013 McKinsey piece. Treat them as folklore. The sturdier figure:

5–15%

revenue lift from getting personalization right (McKinsey)

2013

origin of the Amazon 35% / Netflix 75% folklore — contested since

mid-2026

the year the "ChatGPT moment" stopped being theoretical

01

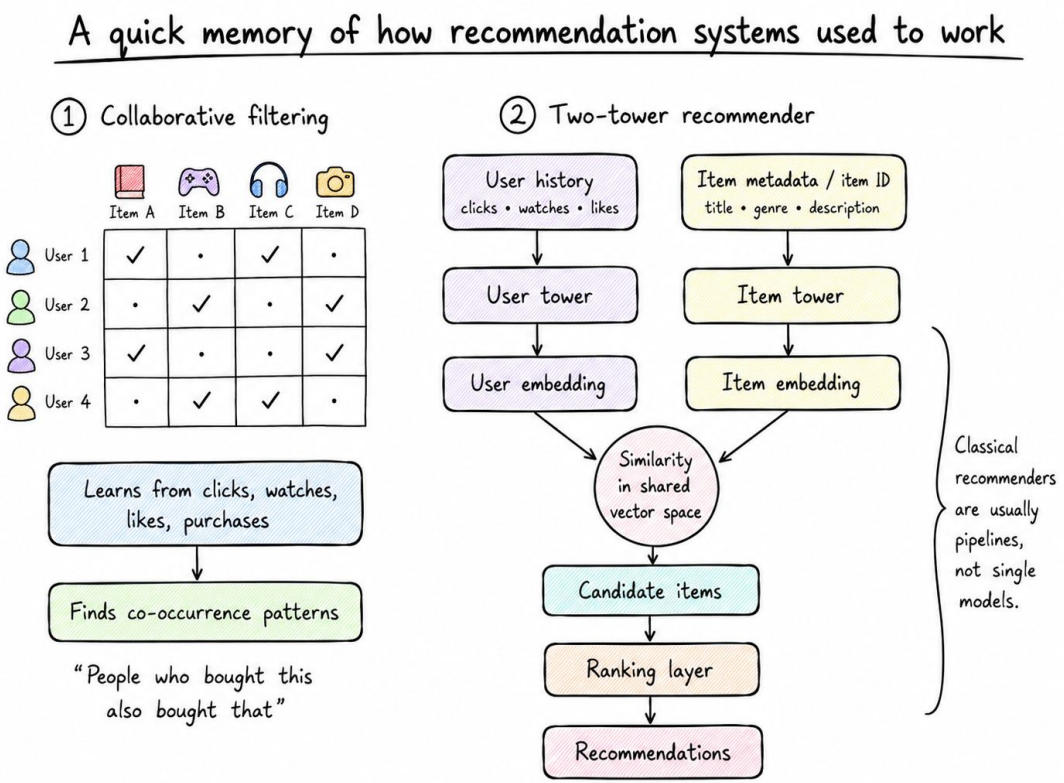
How recommendation used to work?

FOUNDATIONS

Collaborative filtering & the two-tower

Collaborative filtering learns from the interaction matrix: clicks, watches, skips. Brilliant at co-occurrence: *people who bought this…*, but to it an item is just an ID.

The two-tower network scaled this up: encode user and item into one space, then nearest-neighbour search a precomputed index. “Embeddings + a vector DB.”



FOUNDATIONS

Three structural weaknesses

PROBLEM 1

Cold start

A brand-new item has no interactions; a new or anonymous user has no durable history. A behaviour-only model has nothing to stand on.

PROBLEM 2

Weak semantics

It learns what co-occurs, not what something means. Rich text, images, audio and reviews are left largely unused.

PROBLEM 3

Scaling limits

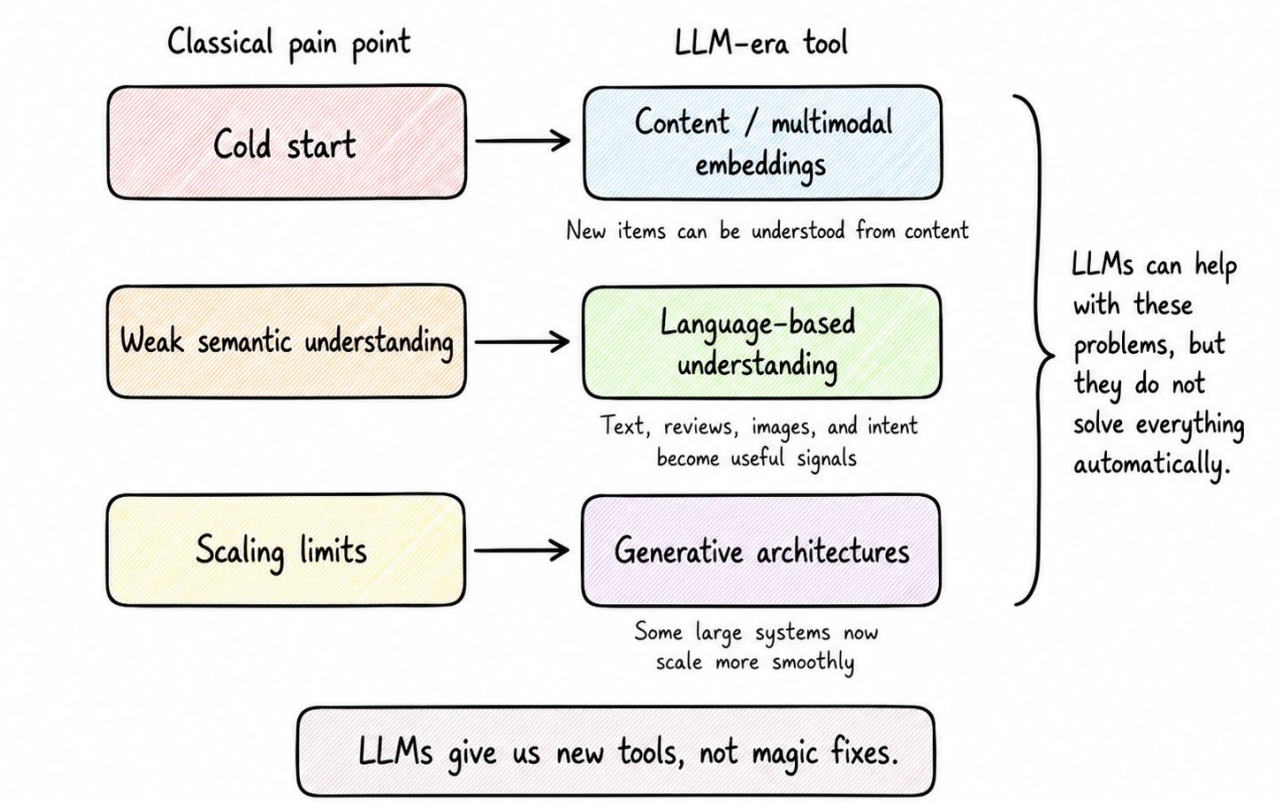
At industrial scale, adding parameters to a classical stack plateaus — unlike language models, where more compute keeps paying off.

02

Where to put the LLM?

THE TURN

What LLMs Change



THE TURN

Two modes, one question: where should the LLM enter?

NON-GENERATIVE · ENHANCEMENT

Improve the recommender

The LLM never writes to the user. It makes embeddings, enriches features, or reranks/scores candidates inside a multi-stage pipeline.

GENERATIVE

Be the recommender

It produces items (or item IDs + rationales) directly: prompting / in-context learning, fine-tuning / LoRA, or generative retrieval over a controlled vocabulary.

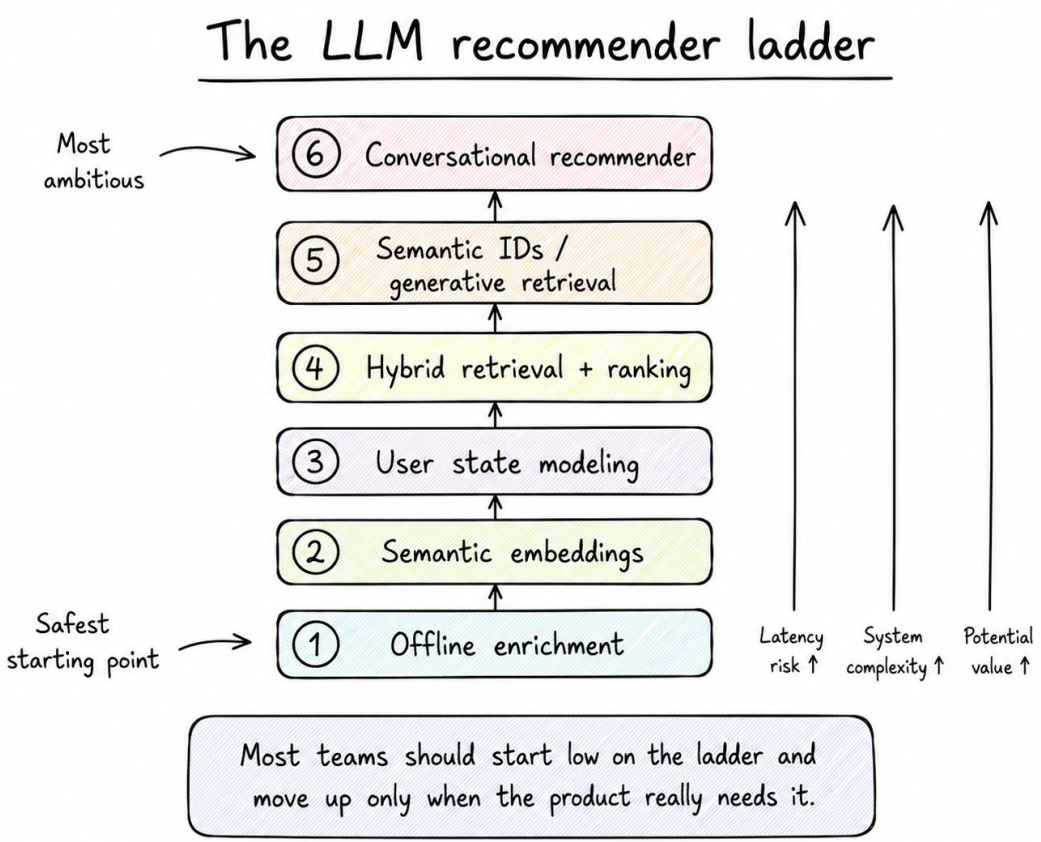
THE LADDER

Climb from safest to most ambitious

Here is the ladder we will climb.
Read it from the bottom up:

- the safest options are at the bottom
- the most ambitious (risky) at the top

Most teams should start at pattern 1 or 2 and only climb higher when the product need justifies the operational cost.

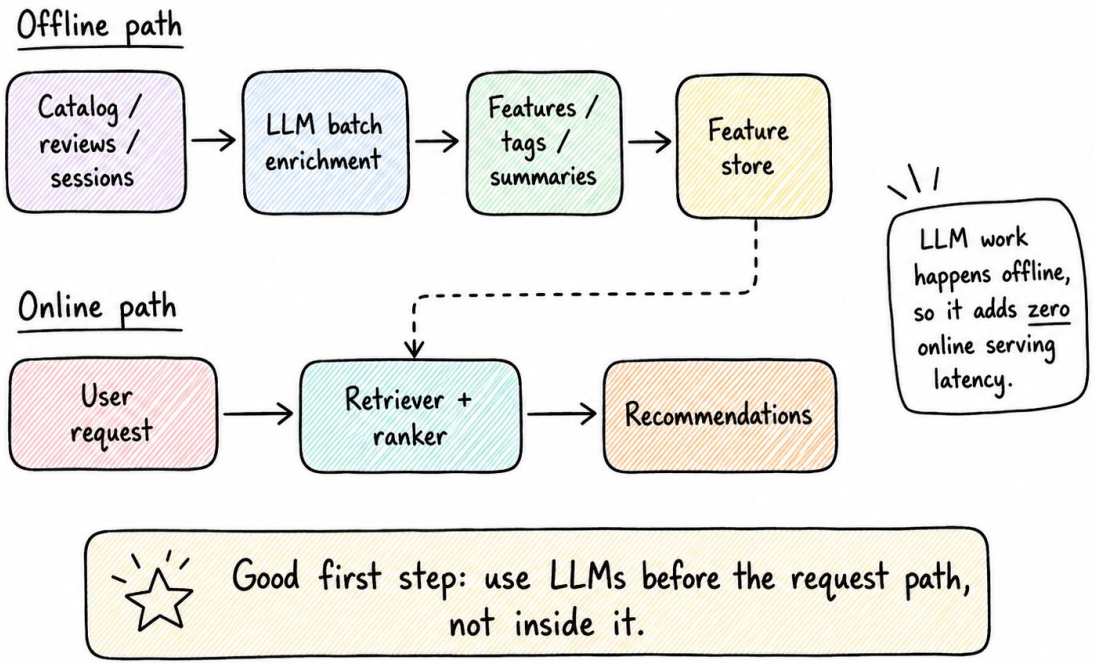


RUNG 1 · OFFLINE

Start offline, where nothing pages you at 2 a.m.

Run the LLM in a batch job: summarize reviews, extract attributes, tag mood, normalize messy descriptions. The output is just more features for the model you already trust.

Zero online serving latency; the biggest objection evaporates. The trade-off moves to batch cost, freshness and drift. Best risk-to-reward to start.



RUNG 2 · EMBEDDINGS

Let meaning become the embedding

Encode an item's title, synopsis, genres and tags into a vector. Similar items land near each other by meaning; a new item gets a sensible position on day one, before any clicks.

Content embeddings ask: what is this about?

Interaction embeddings ask: what do people do with it?

You usually need both.

RUNG 2 · EMBEDDINGS

Encoding meaning with sentence-transformers

movielens_llm_recsys.ipynb

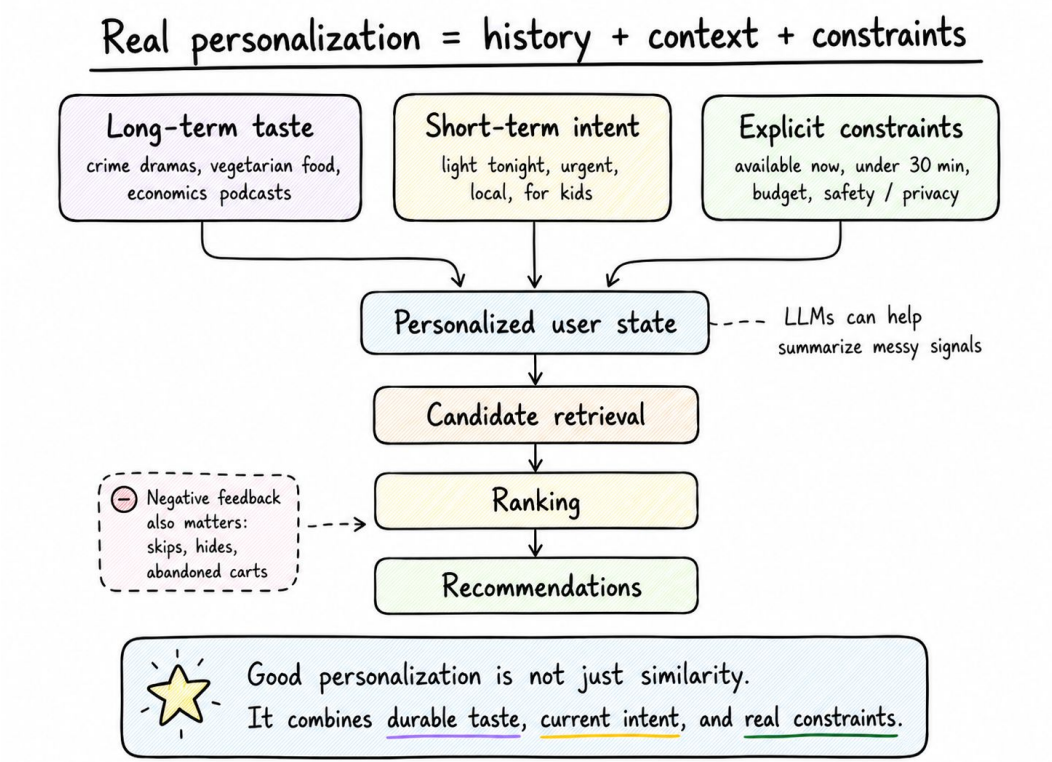
```
from sentence_transformers import SentenceTransformer

# one short "document" per movie: title + genres + tags
docs = [f"{t}. genres: {g}. tags: {k}" for t, g, k in movies]

model = SentenceTransformer("all-MiniLM-L6-v2") # 384-dim, CPU-friendly
item_emb = model.encode(docs, normalize_embeddings=True)

item_emb.shape # -> (n_movies, 384)
```

RUNG 3 · THE USER

Model the user as history and state

Real personalization is a model of the person, the moment and the constraints: long-term taste + short-term session intent + explicit limits.

An LLM can compress messy signals into a compact state but don't blur everything into one mystical vector no one can debug.

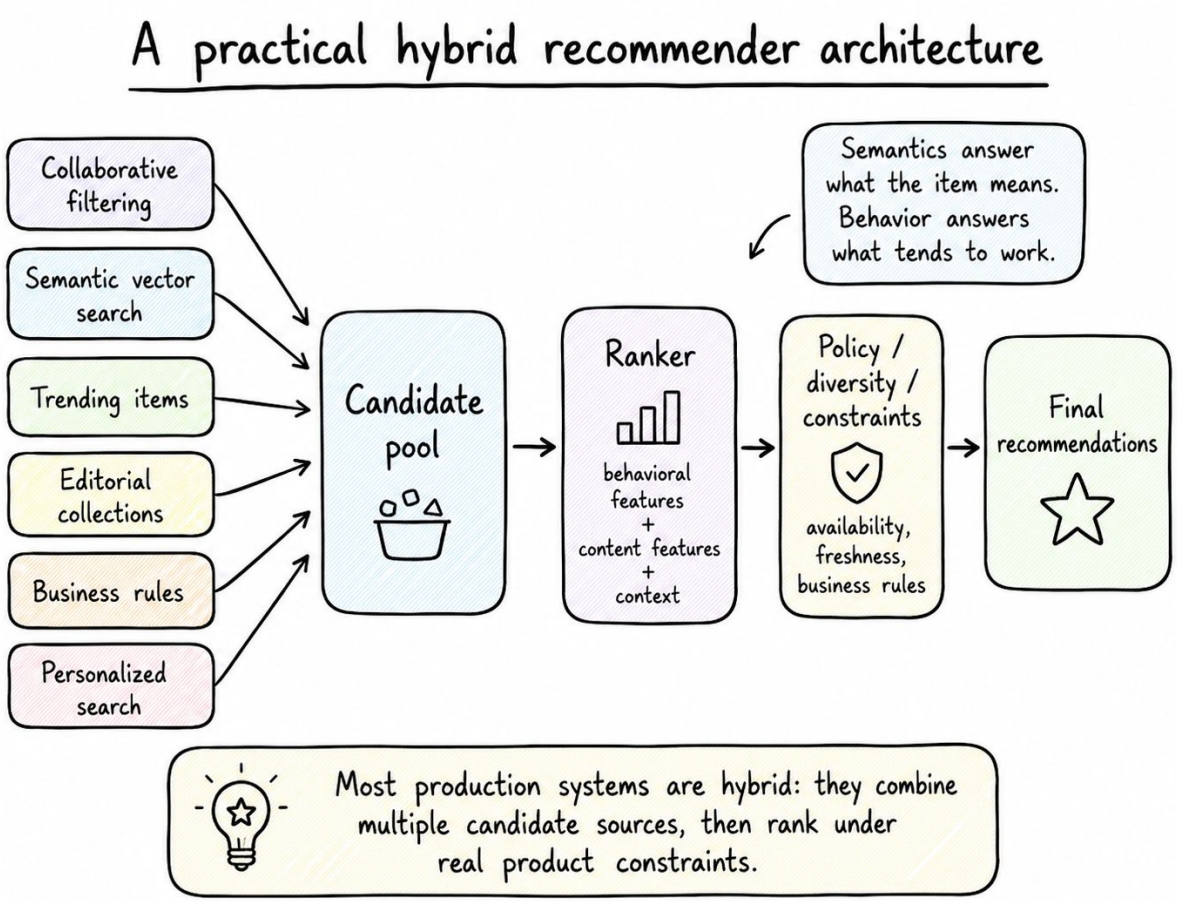


RUNG 4 · HYBRID

Go hybrid

Retrieve candidates from several sources (CF, vector search, trending, rules), then rank with behavioural + content + context features under real policy constraints.

Semantics answer what an item means; behaviour answers what tends to work. Most teams should expect to run this hybrid for a long time.



RUNG 4 · HYBRID

Vector search + the cold-start demo (FAISS)

movielens_llm_recsys.ipynb

```
import faiss

index = faiss.IndexFlatIP(item_emb.shape[1]) # cosine via normalized dot
index.add(item_emb)

# a movie that ISN'T in the dataset — described in one sentence
q = model.encode(["astronauts cross a wormhole to save humanity"], normalize_embeddings=True)
scores, ids = index.search(q, k=5)
# -> Interstellar, Arrival, Contact ...  (no interaction history)
```

RUNG 4 · HYBRID

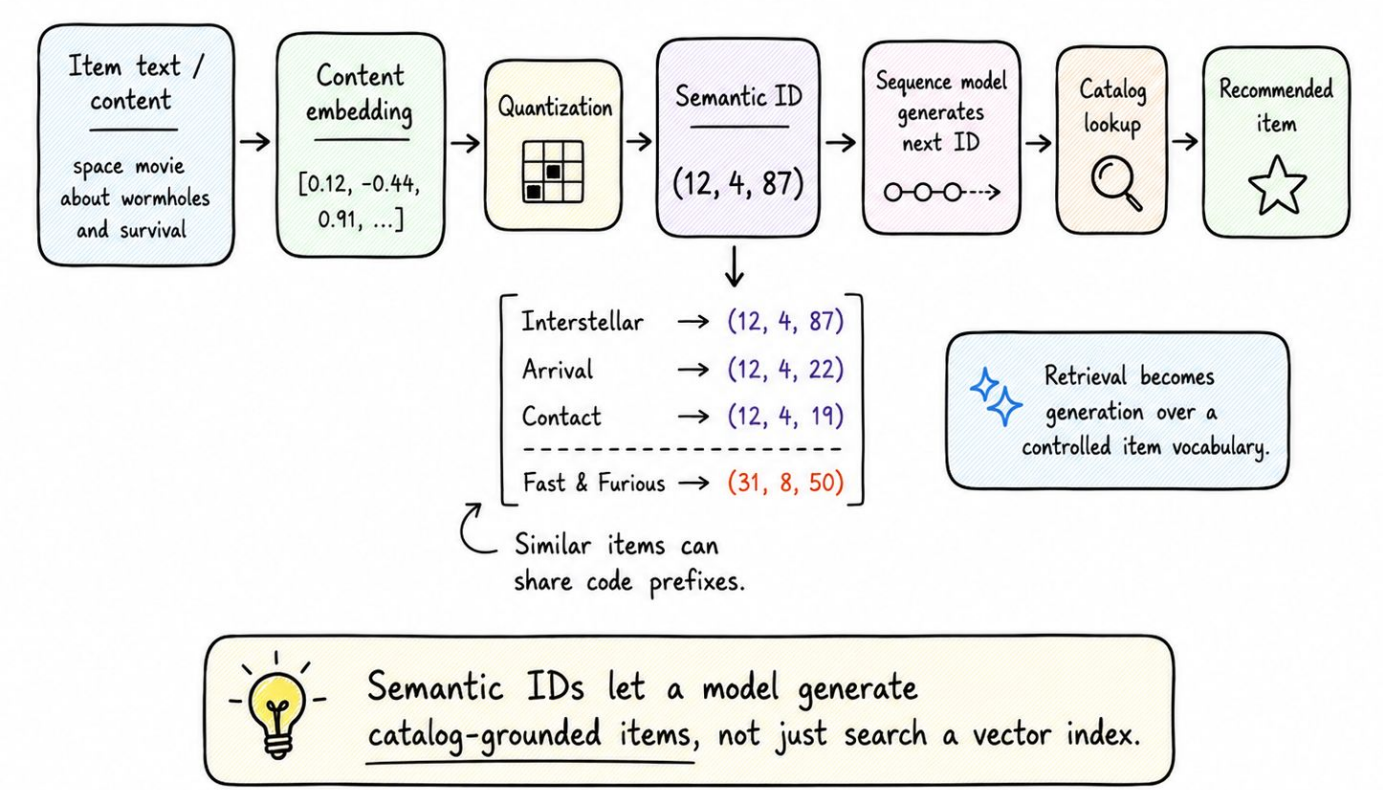
Blending meaning and behaviour with one knob

movielens_llm_recsys.ipynb

```
# semantic similarity to the user's seed item
sem = item_emb @ item_emb[seed]
# collaborative similarity (from SVD item factors)
cf  = item_factors @ item_factors[seed]

alpha = 0.3 # 0 = pure behaviour, 1 = pure meaning
score = alpha * sem + (1 - alpha) * cf
ranking = score.argsort()[::-1]
```

RUNG 5 · GENERATIVE

Turn items into a vocabulary: Semantic IDs



RUNG 6 · CONVERSATIONAL

And finally, let people talk to it

RAG

Grounded generation

The model must not invent items. Ground every suggestion in the real catalog, respect availability, and explain why.

INTENT

Language as the interface

“Something funny but not dumb, like The Good Place, but darker.” Useful when intent is hard to express with clicks.

CAUTION

Most exposed rung

Latency, cost, hallucination, privacy and evaluation all bite hardest here. An interaction layer on top of grounded retrieval — not a free-roaming chatbot.

03

Does the LLM actually help?

THE EXPERIMENT

Five arms, one public dataset, one metric

MovieLens (~100k ratings). Hold out each user's most recent liked movie; rebuild CF on train only; score every arm with Recall@10 / NDCG@10.

Popularity	CF	Content	Hybrid	LLM
what's popular	item-item SVD	MiniLM embedding profile	content + CF (alpha)	generative model as re-ranker

The LLM arm sits where an LLM is cheapest & safest: it reorders a real CF candidate set, it never invents items.

THE EXPERIMENT

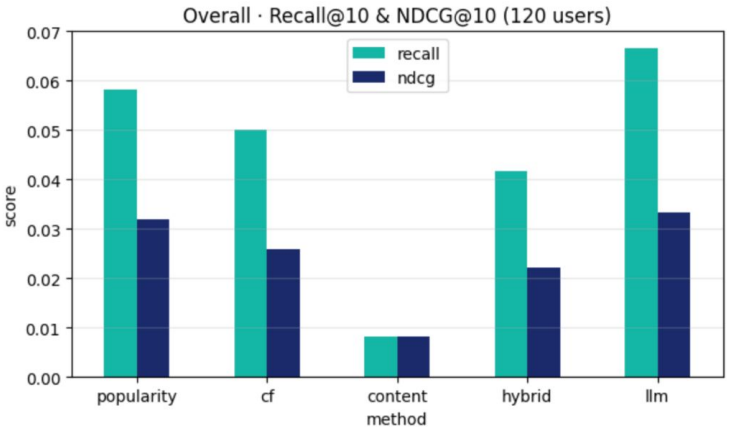
The LLM arm: a grounded reranker

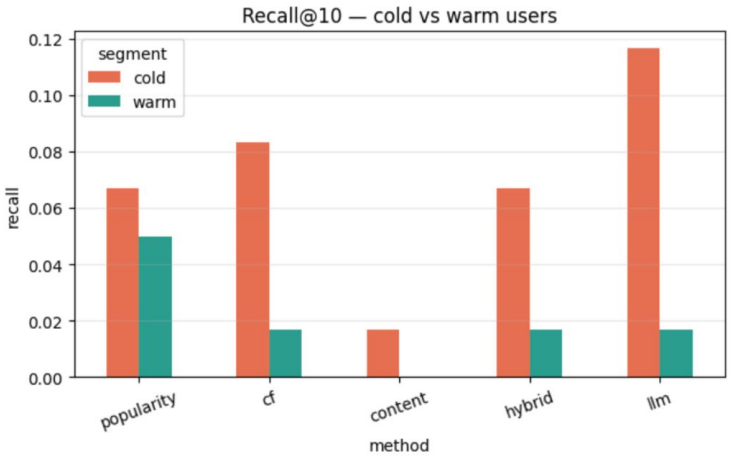
movielens_llm_recsys.ipynb

```
# fast classical retriever gives the candidates;
# the LLM only REORDERS them
cands  = cf_top_n(user, n=40)
prompt = build_prompt(liked_titles, [title_of[c] for c in cands])
ranked = parse_ids(call_llm(prompt)) # OpenAI / Anthropic / Gemini

# ground + back-fill: never invent, always a full top-k
top = [cands[i] for i in ranked] + [c for c in cands if c not in ranked]
top = top[:10]
```

THE VERDICT

It’s not the average, it’s the split





LLM rerank wins on cold users.

Content alone is the weakest arm.

On warm users, popularity leads.

04

The year scaling laws arrived

PATTERN · NATIVE

“Actions Speak Louder than Words” (2024)

Reframe recommendation as sequential transduction; model the user’s stream of actions like a language. Built for high-cardinality, streaming data.

1.5T

parameters; scaling held in a deployed billion-user system

+12.4%

online A/B improvement vs the production baseline

5–15×

faster serving than FlashAttention-2 on long sequences

THE SCOREBOARD

The wave broadened; from one paper to production

System	What shipped	Signal
Meta · GEM (2025)	LLM-scale ads foundation model — a “central brain” for ads recommendation	Lifts ad conversions on IG / FB
Netflix (2025→26)	One foundation model replacing a model zoo; now folded into homepage & search	300M+ members; intent prediction
Spotify (2025)	Semantic-ID generative retrieval for podcast discovery, under prod constraints	Catalog-grounded generation
Kuaishou · OneRec (2025)	End-to-end generative recommender — collapses retrieve-then-rank into one model	≈ 1/10 the operating cost

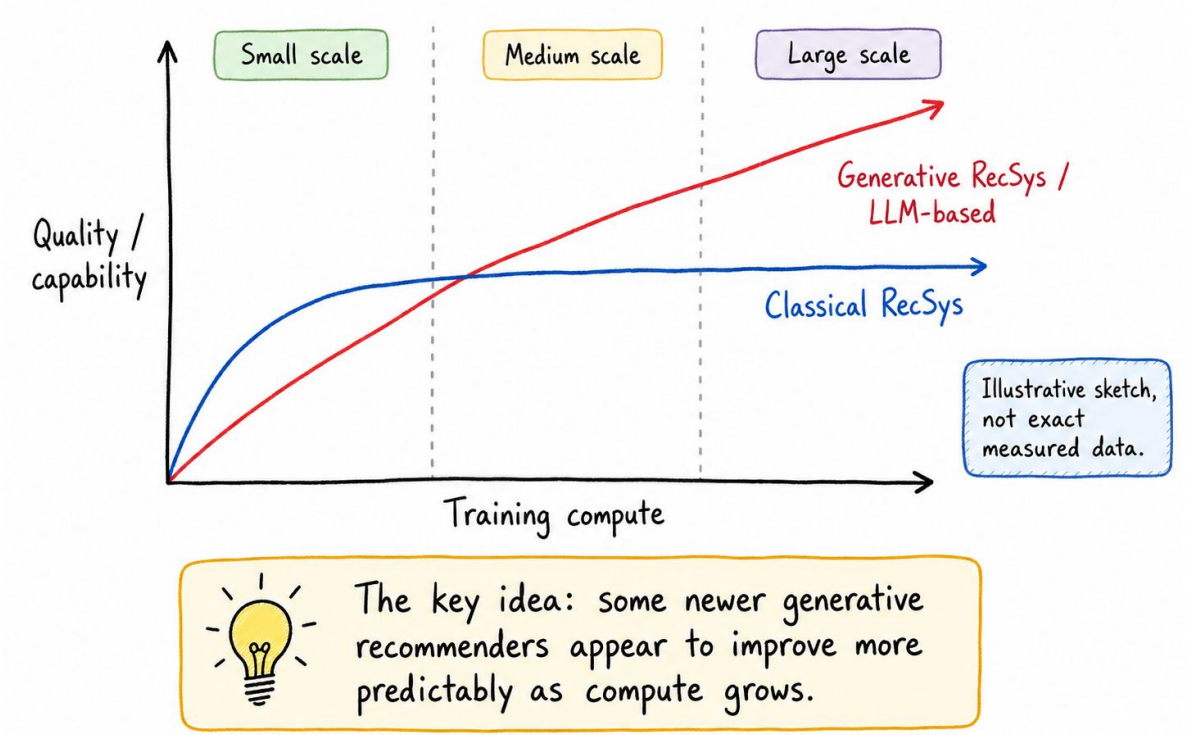
Read these as company-published evidence at extraordinary scale not universal laws.

Most teams aren't near this frontier, and that's fine.

WHY IT MATTERS

Recommendation finally has a scaling law

Some generative recommenders improve predictably as compute grows, where classical stacks plateau.



05

Where LLMs quietly lose

FAILURE MODES

Different patterns, different operational risk

LLM pattern / use case	Latency risk	Cost	Grounding risk	Eval risk
Offline enrichment	Low	Low	Low	Low
Semantic embeddings	Low	Low	Medium	Medium
Hybrid retrieval	Medium	Medium	Medium	Medium
Generative retrieval / Semantic IDs	High	High	High	High
Conversational recommender	High	High	High	High

The higher you go in the stack, the more cost, latency, and evaluation risk you usually carry.

Practical value often starts lower in the stack, not at the chatbot layer.

FAILURE MODES

Two traps worth naming out loud

GROUNDING IS NOT OPTIONAL

It can’t suggest what you don’t have

A recommender can't suggest an unavailable film, an out-of-stock product, or an episode the platform doesn't carry.

- Constrain to the real catalog
- Check availability & policy
- Separate explanation from decisioning
- Log enough to debug why it appeared

POSITION BIAS (LLM-SPECIFIC)

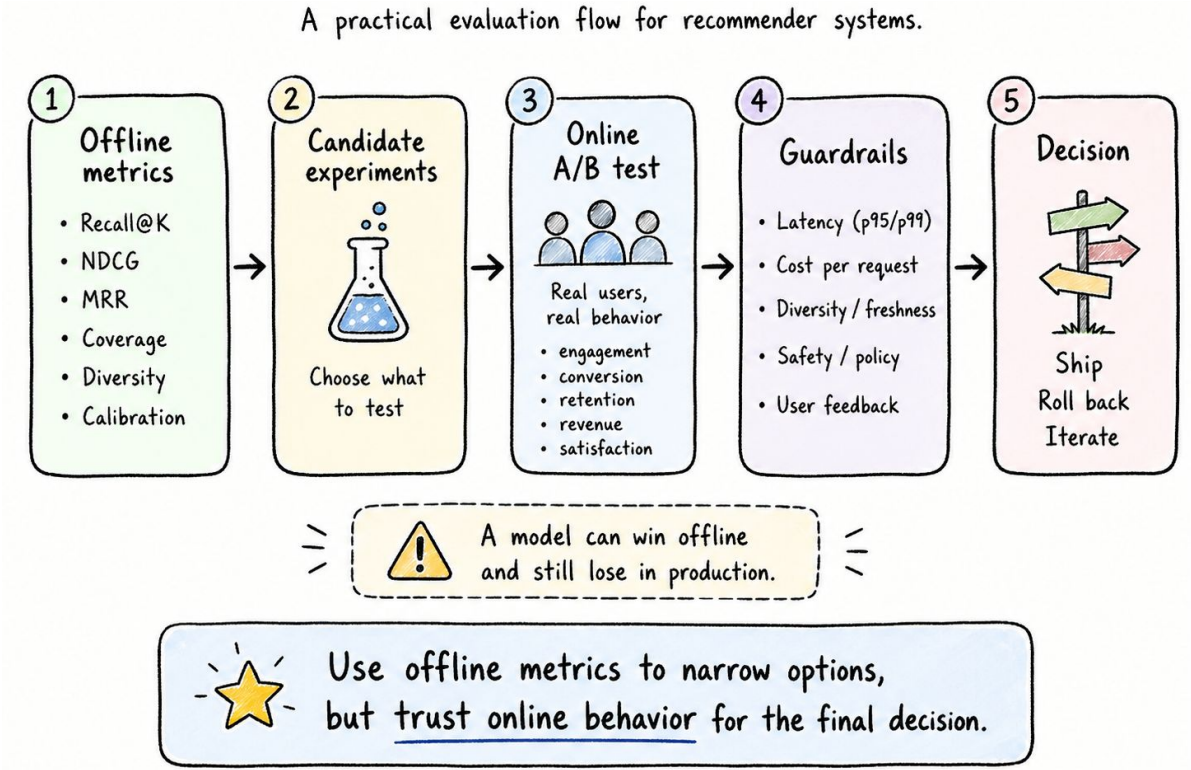
Reverse the list, get a new answer

Ask an LLM to rank a list, then reverse the list — you often get a different answer. It reacts to where an item sits as much as to what it is.

- “Ignore the order” barely helps
- Score candidates independently
- Or pick one at a time (iterative)
- You only catch it if you test for it

EVALUATION

Offline metrics choose experiments. Online behavior decides truth.

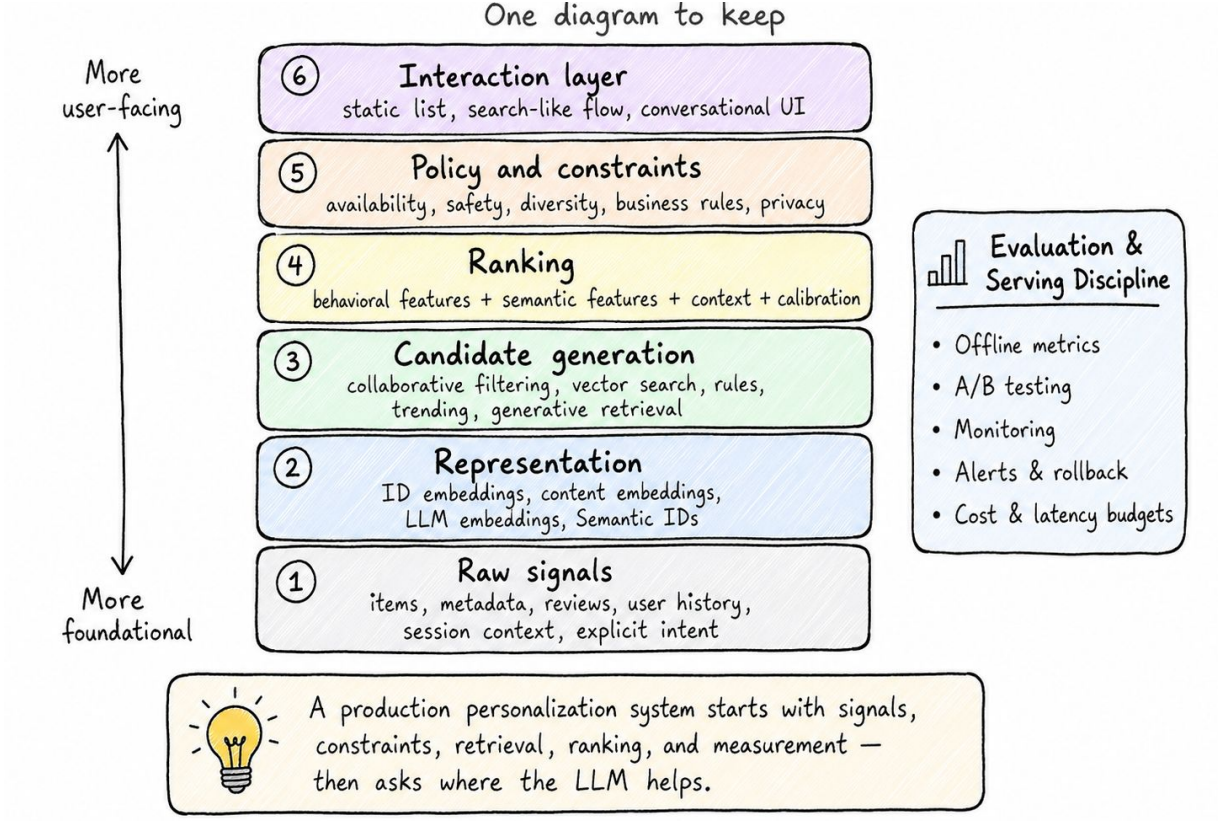


06

Putting it together

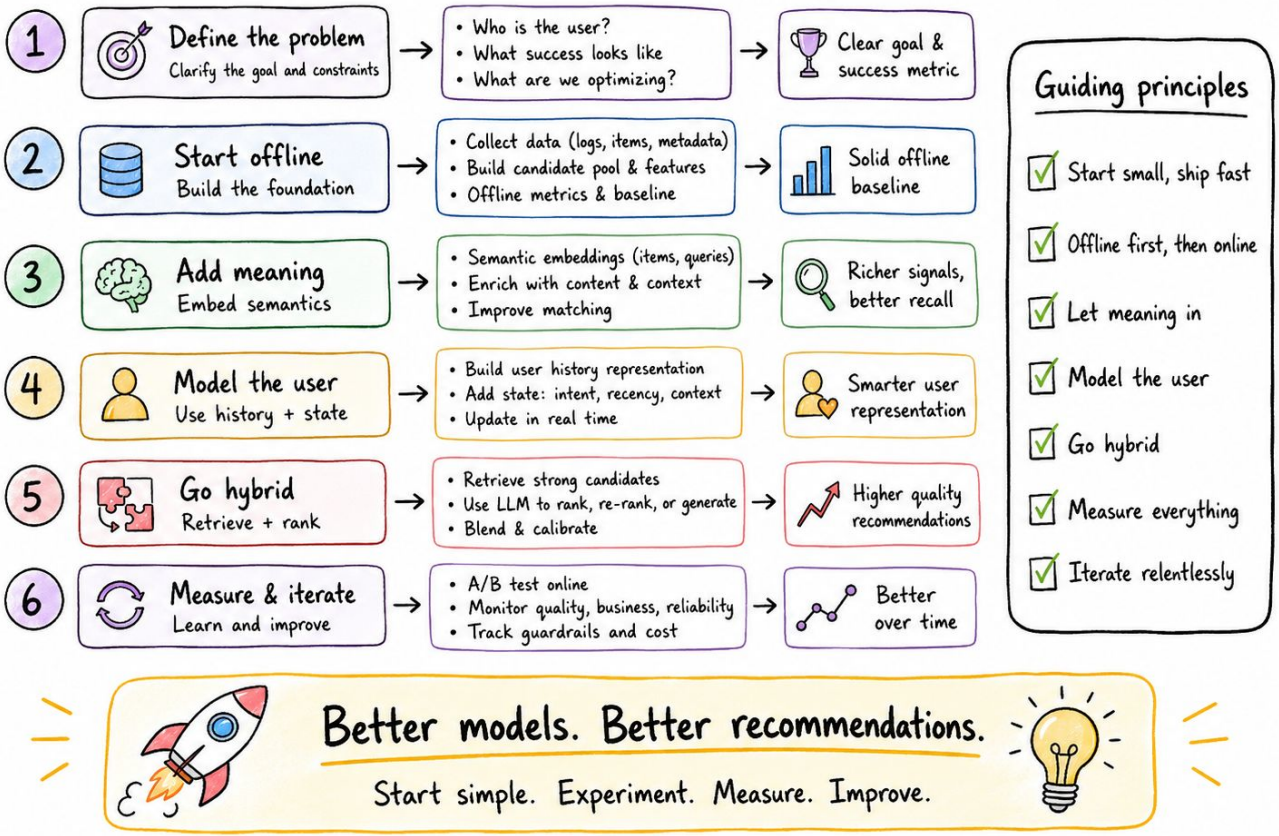
ONE DIAGRAM TO KEEP

The production LLM recommender stack



PLAYBOOK

A playbook for Monday morning



TRY IT YOURSELF

Build all of it on a laptop (MovieLens, CPU)

STEPS 1–2

Baseline → meaning

Item-item CF via truncated SVD, then sentence-transformers embeddings. See CF struggle on cold start.

STEPS 3–4

Search → LLM rerank

FAISS cold-start demo, the alpha knob, and a real generative LLM reranking CF candidates.

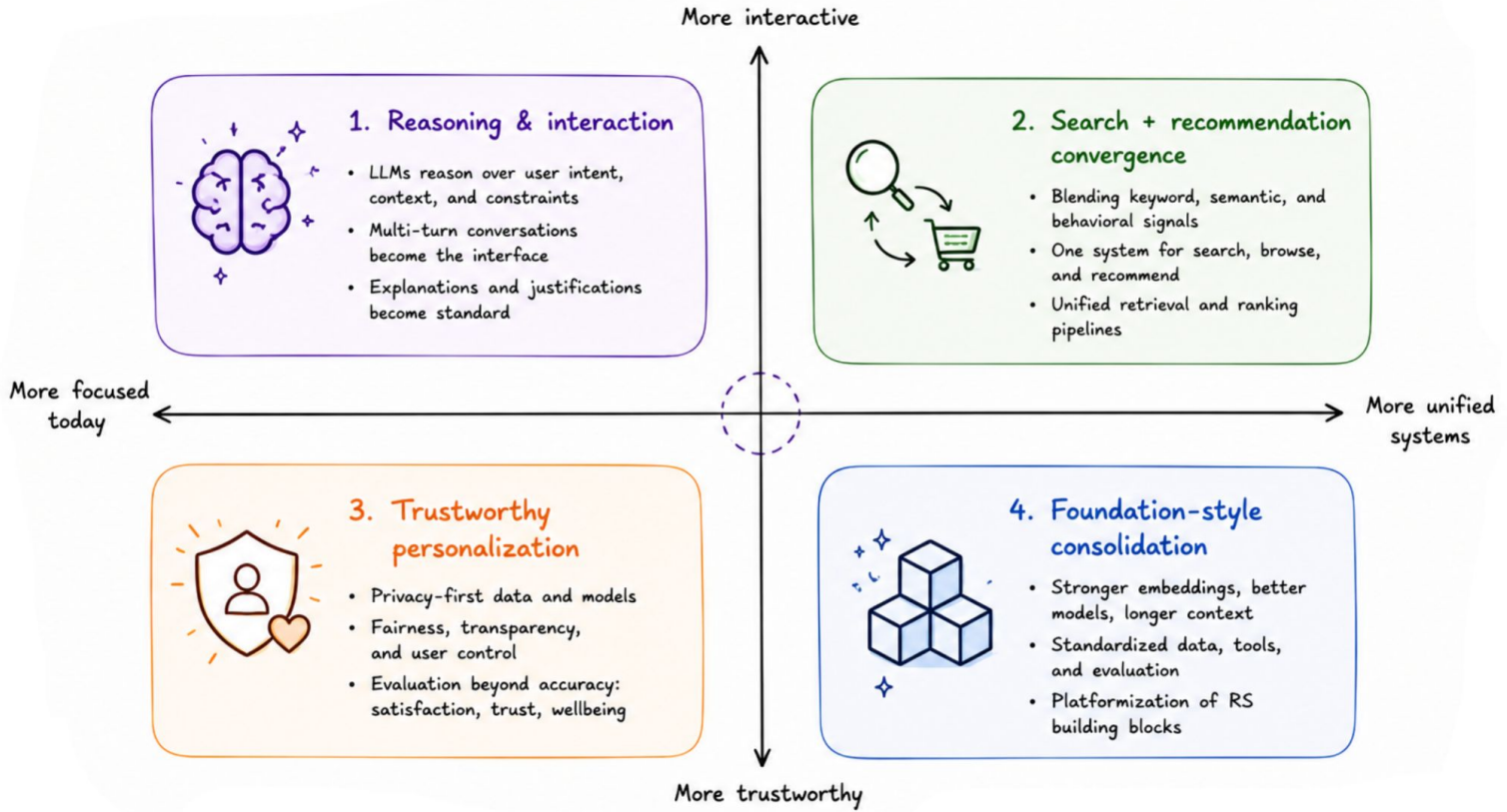
STEPS 5–6

Verdict → eval

Cold-vs-warm Recall@K / NDCG@K + a position-bias test. Scale up via RecBole, Merlin, OneRec.

WHERE THIS IS HEADING

Four trajectories



The mental model to leave with

1. Most recommenders are pipelines — decide which rung the LLM belongs on.
2. Start offline; add semantic embeddings where cold start hurts; go hybrid.
3. Scaling laws are real at the frontier — most teams need the lessons, not the trillion params.
4. Respect latency and cost; ground every generation; test for position bias.
5. Offline metrics choose experiments; an A/B test on real users decides truth.

Thank you!

Özge Çinko · @ozgecinko

Medium Blog Post

GitHub Repo

Linkedin